

Database Course

Indexes for Speeding Up Queries

Lotfi NAJDI

Polydisciplinary Faculty of Taroudant

2025-2026

What is a Database Index?

An index is a **data structure** that improves the speed of data retrieval operations.

This is another object in the database that references the data in tables.

Indexes speed up SELECT queries but slow down INSERT/UPDATE/DELETE

A Simple Analogy

Think of an index as the **index** at the back of a book.

- Without an index: You'd have to read every page to find a topic
- With an index: You jump directly to the page number
- Database without index: Scans every row (FULL TABLE SCAN)
- Database with index: Jumps directly to matching rows

So, Why Use Indexes?

Indexes are essential for improving database performance:

- **Faster Query Performance:** Significantly reduces the time taken to retrieve data.
- **Efficient Data Access:** Optimizes search operations, especially on large datasets.
- **Reduced I/O Operations:** Minimizes the amount of data read from disk.
- **Improved Join Performance:** Speeds up join operations between tables.

Full Table Scan

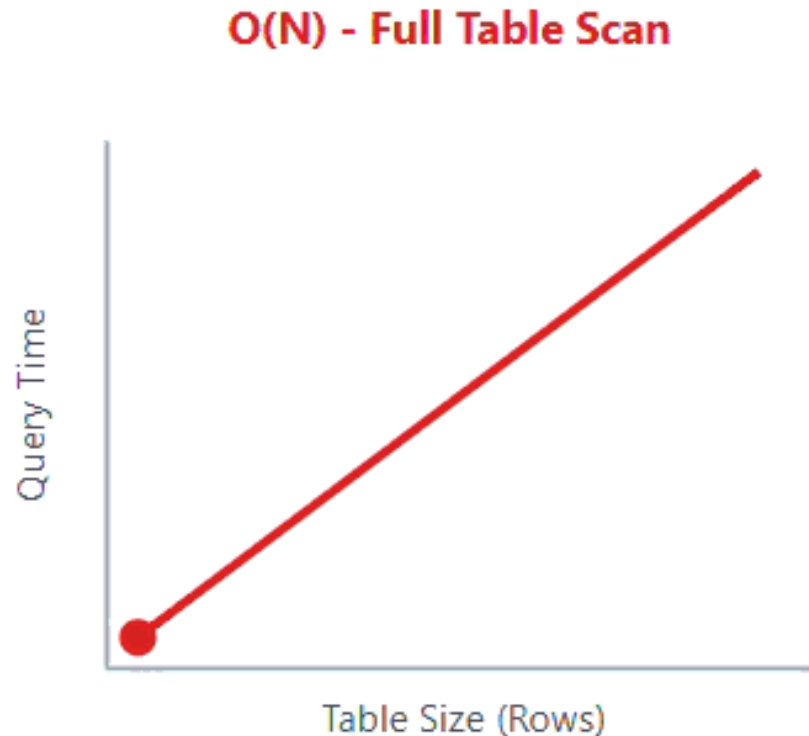
```
SELECT * FROM users WHERE age = 42
```

id	username	age
1	alex	25
2	ben	51
3	zoe	42
4	charlie	38
5	diana	19
6	ethan	30
7	fiona	42
8	george	22
...	...	...

Slow: Scans every row

- This is a Full Table Scan.
- Databases do this by default.
- It's a huge performance bottleneck for large tables.

Performance Cost: Linear Growth



- Full Table Scan performance is directly proportional to table size
- $O(N)$ complexity
- Double the data = double the wait time
- Linear scaling is the enemy of fast applications

The Solution: The Database Index

age_index (sorted)

1. Find all '42' entries

age	pointer
19	→[5]
22	→[8]
25	→[1]
30	→[6]
38	→[4]
42	→[3]
42	→[7]
51	→[2]
...	...

users table

2. Jump to rows [3] & [7]

id	...	age
[1]	...	25
[2]	...	51
[3]	...	42
[4]	...	38
[5]	...	19
[6]	...	30
[7]	...	42
[8]	...	22

- An index is like a book's index: a separate, sorted structure that points directly to the data's location.
- Instead of scanning the whole table, RDBMS uses the index to find all matching rows instantly.

Performance Gain: Logarithmic Growth

The Golden Trade-Off

Indexes are not free. They create a fundamental trade-off.

- **FASTER Reads (SELECT):** Queries on indexed columns are dramatically faster.
- **SLOWER Writes (INSERT, UPDATE, DELETE):** Postgres must do extra work to update the index too.

How Indexes Work Internally

Physical Storage of Indexes

Indexes are stored as **separate data structures** on disk, independent of the table data.

Each index occupies its own space and requires maintenance on every data modification.

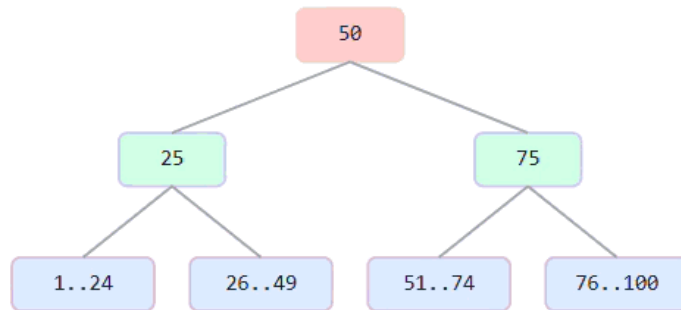
PostgreSQL uses **pages** (8KB blocks) to store both table data and index data.

B-Tree Index Structure

B-Tree (Balanced Tree) is the default and most versatile index type in PostgreSQL.

The Structure of an Index: The B-Tree

- **Root Node:** Entry point
- **Internal Nodes:** Navigation
- **Leaf Nodes:** Point to data



B-Tree search

- Keys are always kept in sorted order within each node.
- This allows for incredibly fast searching.
- Just like finding a word in a dictionary, the database can make a quick decision at each step to narrow down the search.

How B-Tree Search Works

1. **Start at Root:** Compare search value with root node
2. **Navigate Down:** Follow pointer to appropriate child node
3. **Repeat:** Continue until reaching leaf node
4. **Find Data:** Leaf node contains pointer to actual table row

Example: Finding value 30

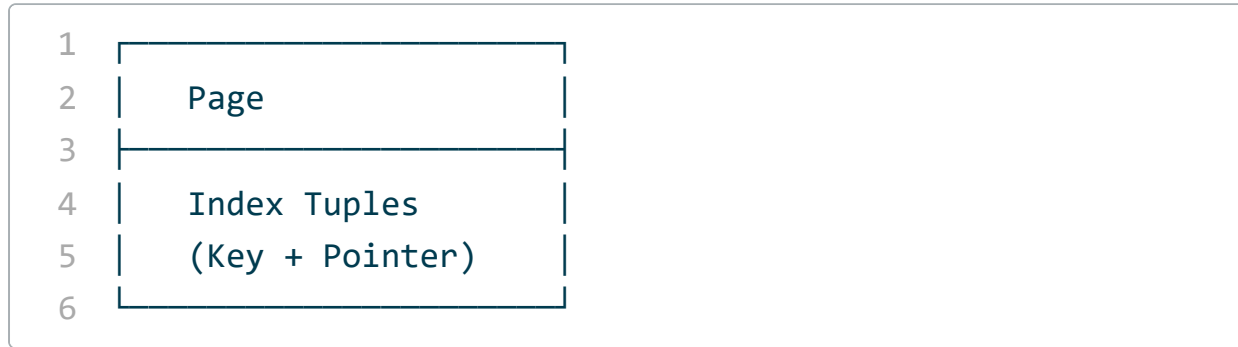
1 Root (50) → Go Left → Internal (25) → Go Right → Leaf [26-49]

Example: Finding value 80

1 Root (50) → Go Right → Internal (75) → Go Right → Leaf [76-100]

B-Tree Page Structure

Each node is stored in one or more **pages**:



- **Keys:** contain the indexed column values like 1, 25, 50, 75
- **Pointers:** point to child nodes or actual table rows

B-Tree Leaf Nodes

Leaf nodes contain the actual pointers to the table rows.

1	
2	Leaf Page
3	
4	Key: 1 → Pointer to Row
5	Key: 2 → Pointer to Row
6	Key: 3 → Pointer to Row
7	...
8	Key: 100 → Pointer to Row
9	

Creating and Verifying Indexe

Single Column Index

The SQL syntax for creating an index is straightforward. A common naming convention is **idx_tablename_columnname**.

```
1 -- For a single column:  
2 CREATE INDEX idx_users_username  
3 ON users (username);
```

Confirm index creation

```
1 SELECT indexname, indexdef
2 FROM pg_indexes
3 WHERE tablename = 'users';
```

Expected Output:

1 indexname	indexdef
2 -----	-----
3 users_pkey	CREATE UNIQUE INDEX users_pkey ON users USING btree (id)
4 idx_users_username	CREATE INDEX idx_users_username ON users USING btree (username)

Primary Keys - Automatic Index

PostgreSQL automatically creates a **unique B-tree index** on primary key columns. For foreign keys and relationships, **indexes are essential but not automatic**.

Multi-Column (Composite) Indexes

You can index multiple columns for queries that filter on several fields at once.

```
1 CREATE INDEX idx_users_fullname  
2 ON users (first_name, last_name);
```

Confirm Composite Index Creation

```
1 SELECT indexname, indexdef
2 FROM pg_indexes
3 WHERE tablename = 'users';
```

Expected Output:

1 indexname	indexdef
2 -----	-----
3 users_pkey	CREATE UNIQUE INDEX users_pkey ON users USING btree (id)
4 idx_users_username	CREATE INDEX idx_users_username ON users USING btree (username)
5 idx_users_fullname	CREATE INDEX idx_users_fullname ON users USING btree (first_name, last_

Dropping Index

```
1 DROP INDEX index_name;
```


Index Maintenance

Dropping unused indexes

```
1 -- Remove an index if it's not being used
2 DROP INDEX idx_customer_last_name;
```

Reindexing for maintenance

```
1 -- Rebuild an index to optimize it
2 REINDEX INDEX idx_customer_last_name;
3
4 -- Rebuild all indexes on a table
5 REINDEX TABLE customers;
```

Unique Indexes

Enforcing data uniqueness

```
1 -- Create unique index to prevent duplicate emails
2 CREATE UNIQUE INDEX idx_customers_email_unique
3 ON customers (email);
4
5 -- This will fail if email already exists
6 INSERT INTO customers (email, name)
7 VALUES ('existing@email.com', 'John Doe');
```

Performance Tips

-  **Index foreign keys** - JOIN operations will be faster
-  **Index WHERE clause columns** - Especially in frequent queries
-  **Index ORDER BY columns** - Sorting will be faster
-  **Monitor index usage** - Remove unused indexes
-  **Consider composite indexes** - For multi-column WHERE clauses

Real-World Example

```
1  -- Products table with proper indexes
2  CREATE TABLE products (
3      product_id SERIAL PRIMARY KEY,  -- Automatic index
4      name VARCHAR(255),
5      category_id INTEGER,
6      price DECIMAL(10,2),
7      created_date DATE,
8      status VARCHAR(20)
9  );
10
11 -- Strategic indexes
12 CREATE INDEX idx_products_category ON products (category_id);
13 CREATE INDEX idx_products_price ON products (price);
14 CREATE INDEX idx_products_status_date ON products (status, created_date);
```

Common Mistakes to Avoid

-  **Creating too many indexes** - Slows down writes
-  **Not monitoring index usage** - Wasted storage space
-  **Wrong column order in composite indexes** - Put most selective column first
-  **Indexing everything** - Each index has overhead

Query Optimization

The Explain Command

EXPLAIN shows how PostgreSQL will execute a query - the **query execution plan**.

```
1 EXPLAIN
2 SELECT * FROM users WHERE username = 'john_doe';
```

Expected Output:

```
1          QUERY PLAN
2  -----
3  Index Scan using idx_users_username on users  (cost=0.29..8.31
4    rows=1 width=128)
5    Index Cond: (username = 'john_doe')
6  (1 row)
```

Anatomy of a Query Plan

```
1          QUERY PLAN
2  -----
3  Index Scan using idx_users_username on users  (cost=0.29..8.31
4    rows=1 width=128)
5    Index Cond: (username = 'john_doe')
6  (1 row)
```

Each line describes an operation. Let's break down the key parts:

- **Node Type:** The type of operation (e.g., Index Scan)
- **Cost:** The planner's estimate of the work required
 - PostgreSQL doesn't measure cost in milliseconds — it uses relative units so it can compare plans independently of hardware speed. Each action (CPU step, disk read, etc.) has a weight:
- **Rows:** The estimated number of rows this operation will return
- **Width:** The average size (in bytes) of each row

Decoding the 'Cost'

- **Startup Cost:** The cost before the first row can be returned
- **Total Cost:** The estimated total cost to execute the operation

Common Node Types

- Seq Scan (Sequential Scan) Reads the entire table row by row. Used when no suitable index exists or when most rows need to be read anyway.
- Index Scan Uses an index to locate matching rows directly, fetching only the necessary tuples from the table.
- Nested Loop Join Iterates over rows from one table and, for each row, searches for matching rows in another table. Efficient for small datasets or when the inner table is indexed.

Common Node Types

- Aggregate / Group / Filter / Limit / Sort Additional operations applied after scanning or joining — used for grouping, filtering, ordering, and limiting results.
- Hash Join Builds a hash table from one input (the smaller one) and probes it with rows from the other input — very efficient for large, unindexed joins.
- Merge Join (Optional addition) Joins two sorted datasets efficiently by walking through both in order.

The Explain Analyze Command

EXPLAIN ANALYZE actually runs the query and shows real execution time.

```
1 EXPLAIN ANALYZE
2 SELECT * FROM users WHERE username = 'john_doe';
```

Expected Output:

```
1 -----
2  Seq Scan on users  (cost=0.00..180.00 rows=1 width=128)
3                    (actual time=0.234..12.456 rows=1 loops=1)
4    Filter: (username = 'john_doe')
5    Rows Removed by Filter: 9999
6 Planning Time: 0.123 ms
7 Execution Time: 12.678 ms
```

Reading the **ANALYZE** Output

```
1 -----  
2 Seq Scan on users (cost=0.00..180.00 rows=1 width=128)  
3 (actual time=0.234..12.456 rows=1 loops=1)  
4 Filter: (username = 'john_doe')  
5 Rows Removed by Filter: 9999  
6 Planning Time: 0.123 ms  
7 Execution Time: 12.678 ms
```

- **actual time**: Time taken for this operation during execution
- **rows**: Actual number of rows returned
- **loops**: How many times this operation was executed (for nested loops)
- **Rows Removed by Filter**: Number of rows filtered out

When to Use Explain vs Explain Analyze

- Use **EXPLAIN** when you want to see the estimated plan without executing the query. This is useful for understanding how PostgreSQL intends to execute a query and for planning optimizations without affecting the database state.
 - **Example:** You might use it before making changes to a query to see if those changes will improve performance.
 - **Note:** The output is based on statistics and may not reflect the actual execution.
- Use **EXPLAIN ANALYZE** when you want to see the actual execution plan along with real performance metrics. This is helpful for diagnosing performance issues and verifying if the query is running as expected.

Plan 1: Before Index (Slow)

```
1 EXPLAIN ANALYZE
```

```
2 SELECT * FROM users WHERE username = 'john_doe';
```

```
1 Seq Scan on users (cost=0.00..180.00 rows=1 width=128)
```

```
2 (actual time=0.234..12.456 rows=1 loops=1)
```

```
3 Filter: (username = 'john_doe')
```

```
4 Rows Removed by Filter: 9999
```

```
5 Planning Time: 0.123 ms
```

```
6 Execution Time: 12.678 ms
```

Sequential Scan: Scanned all 10,000 rows to find 1 match! ⚠️

Plan 2: After Index (Fast)

```
1 CREATE INDEX idx_users_username ON users (username);  
2  
3 EXPLAIN ANALYZE  
4 SELECT * FROM users WHERE username = 'john_doe';
```

```
1 Index Scan using idx_users_username on users  
2   (cost=0.29..8.31 rows=1 width=128)  
3   (actual time=0.045..0.047 rows=1 loops=1)  
4   Index Cond: (username = 'john_doe')  
5 Planning Time: 0.156 ms  
6 Execution Time: 0.089 ms
```

Index Scan: Found the row directly! 🚀

142x faster! (12.678ms → 0.089ms)

Summary

Indexes are powerful tools for query optimization

- Use them strategically on frequently queried columns
- Monitor their usage with EXPLAIN
- Balance read performance vs write overhead
- Remove unused indexes periodically

Remember: Measure, don't guess! 